Comprehensive Technical Analysis of Advanced Coloration and Material Segmentation within the Kenney Asset Ecosystem

1. Introduction: The Intersection of Low-Poly Aesthetics and Technical Granularity

The domain of 3D asset creation for real-time applications has historically been divided into two distinct methodologies: high-fidelity modeling, characterized by dense topology and complex physically-based rendering (PBR) pipelines, and low-poly modular styling, which prioritizes silhouette, readability, and efficiency. The Kenney asset library, alongside its proprietary tooling suite—Asset Forge and Kenney Shape—stands as a paragon of the latter. For 3D artists, technical artists, and indie developers, the "Kenney Style" represents more than a visual aesthetic; it is a standardized workflow optimized for rapid prototyping and performance. However, this optimization introduces specific constraints regarding surface customization. A recurrent challenge for practitioners utilizing this ecosystem is the application of granular color details to specific sections of a model without disrupting the modular efficiency that defines the pipeline.

The query at hand—how to color specific parts of a 3D model using the Kenney library rather than coloring the entire object—touches upon the fundamental architecture of block-based modeling and texture atlasing. Unlike traditional digital sculpting where a mesh can be painted with infinite resolution, Kenney's tools operate on discrete logic: blocks in Asset Forge and voxels in Kenney Shape. The standard operational mode for these tools is monolithic assignment, where a single geometric entity receives a single material definition. To transcend this limitation and achieve specific, localized coloring (such as painting a single face of a cube or isolating a trim detail on a pre-fabricated asset), one must employ a combination of advanced kitbashing techniques, custom asset injection, and post-export mesh manipulation.

This report serves as an exhaustive technical manual and theoretical analysis of these processes. It explores the geometric underpinnings of Asset Forge, the depth-to-mesh algorithms of Kenney Shape, and the UV coordinate systems that govern the Kenney texture atlases. By dissecting these elements, we establish a robust methodology for differentiating and coloring model sections, enabling artists to maintain the speed of the Kenney workflow while achieving a level of custom fidelity typically reserved for bespoke modeling.

1.1 The Philosophy of Modular coloring

To effectively manipulate the color of Kenney assets, one must first internalize the design philosophy of the library. Kenney assets are generally constructed as "kits"—collections of mesh parts that share a common scale, pivot point logic, and, crucially, a common texture atlas. The coloring paradigm is not one of "painting pixels" on a unique UV map, but rather "assigning coordinates" to a shared palette.

When a user seeks to color a specific part of a model, they are technically attempting one of three operations:

1. **Geometric Segmentation:** Physically separating the mesh so that the target area becomes a distinct object capable of holding a unique material slot.
2. **UV Translation:** Moving the texture coordinates of the target faces to a different region of the shared texture atlas.
3. **Overlay Composition:** Introducing new geometry that sits atop the base mesh, obscuring the original surface with a new color (a technique often referred to as "kitbashing details").

The following sections will rigorously examine each of these operations within the context of the specific tools provided by Kenney.

---

2. Asset Forge: Architectural Constraints and Coloring Workflows

Asset Forge represents the central hub for 3D construction in the Kenney ecosystem. It allows users to assemble models using a library of pre-made blocks. The default behavior of Asset Forge is to treat each block as an atomic unit of material assignment. If a user places a "Wall" block, the material system applies color to the entire wall. This section details how to circumvent this behavior to achieve granular coloring.

2.1 The Native Material System

The material system in Asset Forge is designed for speed. A "Material" is defined as a data object containing a color value (RGB/Hex) and an optional texture reference. The application of these materials provides the first layer of insight into selective coloring.

When a block is instantiated in the scene, it is linked to a material ID. The user interacts with this system via the Material Tab, typically located on the left-hand interface panel.[1] The process is seemingly binary: select a block, click a material, and the block updates. However, the internal logic allows for more complex interactions if one understands the "Block" definition.

Block Atomicity

A "Block" in Asset Forge is a reference to an underlying OBJ file. If that OBJ file defines a single mesh group, Asset Forge can only apply one material to it. This is why a complex block, like a pre-made car chassis, cannot have its tires colored differently from its body within the standard interface—unless the block itself was authored with separation in mind (which many default blocks are not).

To color specific parts natively, the artist must adopt a **"Constructive Solid Geometry" (CSG) mindset**, albeit additive rather than subtractive. Instead of taking a block and painting a stripe on it, the artist must build the stripe as a separate object.

2.2 Technique A: Additive Geometric Coloring (Kitbashing)

The most direct method to color a specific section of a model in Asset Forge without leaving the software is through additive geometry, often referred to as "greebling" or "kitbashing details."

The Mechanism:

Since Asset Forge allows blocks to intersect without creating non-manifold geometry errors during the editing phase, users can simulate detailed coloring by placing thin, primitive shapes over the areas they wish to color.

Step-by-Step Implementation:

1. **Analyze the Target Surface:** Identify the specific face or section that requires a unique color (e.g., a window on a solid wall block).
2. **Select a Primitive:** Navigate to the "Primitives" category in the block library. Select a block that matches the topology of the target area (e.g., a generic Cube for a rectangular window, or a Cylinder for a circular port).
3. **Dimensional Scaling:**
   - Place the primitive in the workspace.
   - Use the Scale widgets (the circle gizmos) or the block properties panel to flatten the primitive. For a "paint layer" effect, scale the thickness (usually Z or Y axis depending on orientation) to the minimum viable value (e.g., 0.05 units).
4. **Precision Placement:**
   - Move the flattened primitive to overlap the target surface.
   - **Critical Technical Note:** To avoid "Z-Fighting"—a rendering artifact where two coplanar faces flicker as the depth buffer struggles to determine which is in front—the additive layer must be slightly offset from the base surface. An offset of 0.001 to 0.01 meters is usually sufficient to ensure the "paint" layer renders on top without visible separation.
5. **Material Assignment:**
   - Create a new Material in the Material Tab (e.g., "Window_Blue").
   - Apply this material to the flattened primitive.
6. **Grouping Strategy:**

- Select the base block (the wall).
- Hold Shift and select the additive block (the window).
- Press Ctrl + G (or use the toolbar Group icon) to group these elements.[1]
- **Insight:** Grouping ensures that if the wall is moved or rotated, the specific colored section moves with it, effectively treating the assembly as a single multi-colored object.

This technique is functionally identical to applying a decal, but it utilizes geometry rather than textures, maintaining the sharp, vector-like quality of the low-poly aesthetic.

2.3 Technique B: Custom Block Injection for Segmentation

For users who require specific coloring on repeated complex objects (like a vehicle wheel where the rim and tire must always be different colors), the "Additive" method becomes tedious to repeat. The advanced solution is to author **Custom Blocks** that are pre-segmented.

Asset Forge allows the importation of external OBJ files to extend its library. By preparing these files correctly in a Digital Content Creation (DCC) tool like Blender, users can create "Color-Ready" blocks.[2]

The Custom Block Workflow:

1. **External Preparation (Blender):**
   - Model the object (e.g., a Sword).
   - Separate the mesh into logical color zones. Detach the "Blade" mesh from the "Hilt" mesh.
   - **Export Strategy:** Export the Blade as sword_blade.obj and the Hilt as sword_hilt.obj.
2. **Directory Structure Configuration:**
   - Locate the Asset Forge installation directory.
   - Navigate to the Collections folder.
   - Create a new directory, for example, User_Weapons.
   - Place the OBJ files inside this folder.
   - *Optional:* Create a meta.ini file to define default scale and rotation corrections.[2]

     Ini, TOML
     ```
     [Main]
     scale=1.0
     rotation=0,0,0
     ```
3. **Assembly in Asset Forge:**
   - Restart Asset Forge or refresh the library.
   - The User_Weapons category will now appear.
   - Place the sword_blade block. Assign it a "Metal" material.
   - Place the sword_hilt block. Align it with the blade. Assign it a "Leather" material.

- Group the two blocks.

This method effectively creates a "Multi-Material Block" by treating the components as a composite entity. It is the most robust way to handle repeated assets that require specific internal coloring.

2.4 Analysis of Export Logic and Mesh Merging

The capability to color specific parts extends beyond the creation phase into the export phase. The choice of export settings in Asset Forge fundamentally dictates whether the model can be recolored in a game engine.[1]

The "Merge Blocks" Parameter:

- **Enabled:** Asset Forge combines all blocks into a single mesh (or a few meshes based on material). This optimizes the geometry by removing hidden faces (culling), which is excellent for performance but destructive for editing. If the entire model is merged, the individual "window" primitive created in Technique A becomes physically fused to the "wall," making it difficult to separate later.
- **Disabled:** Asset Forge preserves the individual identity of every block. The exported file (OBJ or FBX) will contain a hierarchy of separate objects (e.g., Mesh_1, Mesh_2, Mesh_3).

Strategic Recommendation:

For artists who plan to finalize coloring in a game engine (Unity/Unreal), it is imperative to export with **Merge Blocks Disabled**. This results in a "scene" of individual parts. In the game engine, the user can then select the specific "Mesh_1" (the window) and change its material property without affecting "Mesh_2" (the wall), granting ultimate granular control.

---

3. Kenney Shape: Depth-Based Color Separation

Kenney Shape operates on a distinct technical paradigm: translating 2D bitmaps into 3D voxel meshes. The coloring workflow here is intrinsic to the creation process, as the "paint" applied to the 2D canvas becomes the material of the resulting 3D faces.[4] However, advanced users can leverage the **Depth** mechanics to create segmentation for post-processing.

3.1 The Pixel-to-Voxel Translation Matrix

In Kenney Shape, the canvas is a grid (up to 64x64 in newer versions) where each cell holds two data points: Color and Depth (Z-height).

- **Color Data:** Dictates the surface material.
- **Depth Data:** Dictates the extrusion height.

To color a specific part of a model, the primary method is simply to paint that pixel on the 2D grid. However, a common issue arises when users want to separate parts for later manipulation (e.g., creating a character where the hat can be swapped).

3.2 Depth-Based Segmentation

If a user wants to ensure that a specific part of the model (e.g., a visor on a helmet) is distinct and easily selectable in external software, they can utilize depth disparity.

Workflow:

1. **Base Layer:** Draw the helmet pixels with a depth of 5.
2. **Detail Layer:** Draw the visor pixels with a depth of 6 (one unit higher).
3. **Result:** The generated mesh will have a physical geometric "step" between the helmet and the visor.
4. **Post-Processing implication:** When this mesh is imported into Blender, the visor faces are coplanar and offset from the helmet. This allows for rapid selection using "Select Linked Flat Faces" or "Select by Normal," facilitating easy material reassignment or separation.

3.3 Color Index vs. Luminosity Depth

Kenney Shape introduces advanced generation modes where depth is calculated based on **Color Index** or **Luminosity** rather than manual depth painting.[4]

- **Color Index Mode:** This mode is powerful for creating "stepped" terrain or machinery. The user defines a palette of colors (e.g., 1 to 8). Color 1 always extrudes to height 1, Color 2 to height 2, and so on.
    - **Differentiation Strategy:** By reserving specific colors in the palette for specific structural heights, the user can enforce a strict separation of parts. For instance, using "Color 8" exclusively for "Rooftops" ensures that all roof geometry is generated at the maximum height, isolating it from the "Wall" geometry generated at "Color 4".

---

4. The Theory and Mechanics of Kenney Texture Atlases

A defining technical characteristic of the Kenney asset library is the extensive use of **Texture Atlases**. An atlas is a large image containing multiple smaller sub-images (textures) packed together. In the context of Kenney's low-poly assets, these atlases often take the form of color grids or palettes. Mastering the atlas is the key to coloring specific parts of existing, pre-made Kenney assets.[6]

4.1 Deconstructing the Color Grid Atlas

The standard Kenney texture atlas (often named colormap.png or pattern.png) is typically a 256x256 or 512x512 pixel image divided into a grid of colored squares.

Structure:

- **Grid Layout:** The image is divided into regions (e.g., 4x4 or 8x8).
- **Color Assignment:** Each square contains a solid color or a texture pattern (e.g., brick, wood) tinted to a specific color.
- **UV Mapping:** The vertices of a 3D model are mapped to coordinates that fall within these squares. A "Red" car door is simply a mesh whose UVs are positioned inside the red square of the atlas.

4.2 The "UV Slide" Technique

To color a specific part of a compiled Kenney model (an OBJ/FBX downloaded from the site), one does not repaint the texture image. Repainting the image would change the color of *every* object in the game that references that red square. Instead, the standard workflow is **UV Translation**.

Workflow:

1. **Import:** Load the asset into a DCC tool (Blender).
2. **Select:** Enter Edit Mode and select the specific faces of the model you wish to change (e.g., the trim of a doorway).
3. **Visual Confirmation:** Open the UV Editor window. You will see the UV island for those faces sitting on top of a specific color block (e.g., Brown Wood).
4. **Translation:** Select the UV island in the editor and translate it (G key in Blender) to hover over a different color block (e.g., Grey Stone).
5. **Result:** The specific part of the model now renders as Grey Stone, while the rest of the model remains Brown Wood.

This technique is non-destructive to the texture file and is the most performant method for game engines, as it does not require creating new material slots or texture files—only modifying vertex data.

4.3 Gradient Atlases and Occlusion

Newer Kenney assets utilize **Gradient Atlases**. Instead of solid blocks, the color squares contain a vertical gradient (Dark to Light).[6]

- **Vertical UV Mapping:** By moving the UVs of a specific mesh part *up* or *down* within the same color square, the artist can change the brightness of that part.
- **Differentiation:** This is used to create specific shading effects. For example, to make the bottom of a wall look dirty or occluded, the artist selects the bottom row of vertices and

moves their UVs down towards the darker part of the gradient block. This "colors" the bottom of the wall with a shadow, without needing a separate shadow map.

---

5. Post-Export Surgical Workflows: Differentiation and Separation

While Asset Forge and Kenney Shape allow for creation, precise specific-part coloring often requires "surgical" intervention in dedicated 3D software. This section provides step-by-step instructions for differentiating parts in industry-standard tools.

5.1 Blender: The Mesh Separation Workflow

Blender is the primary tool for modifying Kenney assets due to its robust mesh editing capabilities.[8]

Scenario: Splitting a Merged Asset

If an asset was exported from Asset Forge with "Merge Blocks" enabled, it arrives in Blender as a single mesh. To color just one section:

1. **Import:** File > Import > OBJ/FBX.
2. **Edit Mode:** Select the object and press Tab.
3. **Selection (L):** Hover the mouse over the specific part you want to color (e.g., a tire). Press L (Select Linked). This selects all connected geometry.
   - *Note:* If the mesh is physically welded, L might select the whole object. In this case, use Face Select mode (3) and C (Circle Select) to paint the selection.
4. **Separation (P):** With the part selected, press P and choose **"Selection"**.
5. **Outcome:** The specific part is now a separate object in the Outliner (e.g., Object.001).
6. **Coloring:** You can now assign a completely new material to this separated object, or move its UVs as described in Section 4.2.

Scenario: The Bisect Tool for Custom Cuts

Sometimes the geometry doesn't exist to separate a color. For example, you want the top half of a pillar to be red and the bottom white, but it's a single long polygon.

1. **Select Geometry:** In Edit Mode, select the pillar faces.
2. **Bisect:** Search for the "Bisect" tool (often nested under the Knife tool).
3. **Draw Cut:** Click and drag a line across the mesh where the color transition should occur.
4. **Material Assignment:**
   - Go to the Material Properties tab.
   - Add a new Material Slot (+).
   - Assign the desired color to the new slot.
   - With the faces above the cut selected, click **Assign**.

5.2 Unity: ProBuilder Integration

For users avoiding Blender, Unity's **ProBuilder** package allows for similar differentiation.[10]

1. **ProBuilderize:** Import the Kenney mesh. Go to Tools > ProBuilder > Object > **ProBuilderize**. This converts the mesh into an editable format.
2. **Face Selection:** Select the "Face" element mode in the ProBuilder toolbar.
3. **Select:** Click the specific faces to color.
4. **Vertex Coloring:** Open the ProBuilder Vertex Color editor. Apply a color (e.g., Red).
   - *Constraint:* This requires a shader that supports Vertex Colors (like the URP Standard or a custom Shader Graph).
5. **Material Detachment:** Alternatively, with faces selected, use the "Detach Faces" command to split them into a new GameObject, which can then receive a standard Material.

5.3 Unreal Engine 5: Modeling Mode

Unreal Engine 5 features a comprehensive **Modeling Mode** (Shift + 5).[11]

1. **TriSel:** Use the **TriSel** (Triangle Select) tool to paint the specific area of the mesh.
2. **Materials:** In the tool options, look for "Material ID". You can assign the selection to a different Material Element (e.g., Change from Element 0 to Element 1).
3. **Accept:** Click Accept. The Static Mesh now has two material slots.
4. **Static Mesh Editor:** Open the asset. You can now drag a different material onto Slot 1, coloring only the selected part.

---

6. Table 1: Comparison of Coloring Strategies by Tool

The following table summarizes the optimal workflow based on the user's starting point within the Kenney ecosystem.

Starting Tool	Objective	Recommended Strategy	Pros	Cons
Asset Forge	Color a specific geometric detail (e.g. window).	**Kitbashing (Additive)**: Place flattened primitive over surface.	Native to tool; visual feedback; easy to iterate.	Increases polycount; risk of Z-fighting if not offset.
Asset Forge	Color a sub-compone	**Custom Block Import**: Import	Cleanest result;	Requires external

	nt of a complex block (e.g. tire of a wheel).	pre-separated OBJ parts.	reusable in library.	modeling (Blender) to prepare blocks.
Kenney Shape	Color distinct vertical layers.	**Depth Separation**: Draw parts at different depth levels.	Creates geometric distinction for easy post-process selection.	alters the silhouette/shape of the model.
Blender (DCC)	Change color of an existing part (e.g. shirt).	**UV Translation**: Move UV islands to different atlas color.	Most optimized; no new textures/materials needed.	Destructive to UVs; requires understanding of UV space.
Game Engine	Runtime color change (e.g. team colors).	**Material ID Splitting**: Separate mesh parts into unique objects/slots.	Allows dynamic material swapping in code.	Increases draw calls (more materials per object).

---

7. Advanced Considerations: File Formats and Metadata

The technical success of selective coloring often hinges on the file format used to transport the model between tools.

7.1 OBJ vs. FBX for Segmentation

- **OBJ (Wavefront):** A text-based format. It supports **Material Groups** (usemtl). If you export an OBJ from Asset Forge, it typically writes material definitions that group faces by color. This is advantageous for importing into Blender, as Blender can automatically split these into selectable material slots.
- **FBX (Filmbox):** A binary format. It supports hierarchy and transforms better than OBJ. If exporting unmerged blocks from Asset Forge, FBX is superior for preserving the

parent-child relationship of the blocks, making it easier to select "that one specific block" in Unity's hierarchy view.[12]

7.2 The meta.ini Configuration

For users creating custom blocks to solve coloring issues, the meta.ini file in the custom collection folder is a crucial, often overlooked feature.[2] It allows the user to normalize the scale of their custom parts.

- *Why this matters:* If you import a "Rim" and a "Tire" from Blender to assemble a colored wheel, they must align perfectly. Ensuring both have scale=1.0 in the meta.ini prevents Asset Forge from auto-scaling them based on bounding box size, ensuring they snap together correctly for the multi-color assembly.

---

8. Conclusion and Strategic Guidance

Coloring specific parts of a 3D model within the Kenney ecosystem is a task that requires navigating the balance between modular convenience and bespoke detail. The user is not limited to "coloring the entire object," but the path to granular coloring requires adopting specific workflows that align with the tools' internal logic.

Strategic Summary:

1. **In Asset Forge:** Do not rely on the bucket fill tool for complex assets. Build complexity through **addition** (placing small colored primitives) or **segmentation** (importing custom split blocks). Always export with "Merge Blocks" disabled if you intend to tweak colors in the game engine.
2. **In Kenney Shape:** Treat the canvas as a depth map as much as a color map. Use depth differences to create physical separation between color zones, facilitating easier selection in post-processing.
3. **With Pre-made Assets:** Master the **Texture Atlas**. Do not repaint texture files; instead, learn to slide UV coordinates. This is the "secret handshake" of efficient low-poly art, allowing for infinite color variations from a single texture source.

By integrating these techniques—geometric kitbashing, depth indexing, and UV manipulation—artists can achieve a high level of chromatic detail while preserving the charm and performance characteristics that make the Kenney library a staple of modern game development.

9. Appendix: Detailed Step-by-Step Instruction Sets

9.1 Asset Forge: Creating a Multi-Colored Custom Block

Objective: Create a reusable "Traffic Cone" block where the base is Orange and the stripe is

White.

1. **Preparation (Blender):**
   - Model a cone.
   - Select the faces for the white stripe. Press P > Selection to separate them.
   - Export the Orange base as cone_base.obj.
   - Export the White stripe as cone_stripe.obj.
2. **Asset Forge Setup:**
   - Go to your Asset Forge directory > Collections.
   - Create folder Custom_Traffic.
   - Copy both OBJs into this folder.
3. **Assembly:**
   - Launch Asset Forge. Select Custom_Traffic category.
   - Place cone_base. Assign Material: Orange.
   - Place cone_stripe. It should snap to the correct height if pivots were aligned in Blender. Assign Material: White.
   - Select both. Press Ctrl + G.
4. **Result:** You now have a single selectable traffic cone with specific part coloring, ready for duplication.

9.2 Blender: UV Sliding for Kenney Assets

Objective: Change a specific roof tile from Red to Blue on a Kenney House model.

1. **Import:** Import house.obj.
2. **Setup:** Open the **UV Editor** pane. Load colormap.png.
3. **Edit:** Select the house. Press Tab for Edit Mode.
4. **Select Face:** Click the specific roof tile face in the 3D view.
5. **Identify UV:** In the UV Editor, locate the dot/island representing that face. It will be on a Red square.
6. **Move:** Press G. Move the mouse until the dot is over a Blue square.
7. **Finalize:** Press Tab to exit. The tile is now Blue. Export the model.

9.3 Kenney Shape: Color Indexing for Terrain

Objective: Create a terrain chunk where water is low (Blue) and grass is high (Green).

1. **Palette:** Load a palette where Color 1 is Blue and Color 2 is Green.
2. **Settings:** In the "Depth" tab (or using numeric keys), ensure depth generation is set to relate to color index (if using advanced settings) or manually select depth.
3. **Draw:**
   - Select Blue. Press 1 (Depth 1). Draw the water.
   - Select Green. Press 2 (Depth 2). Draw the land.
4. **Generate:** The 3D model will have physical height differences matching the colors, ensuring the water is recessed.

5. **Export:** Save as OBJ for use in Unity.

These workflows represent the synthesis of artistic intent and technical workaround, the hallmark of proficient usage of the Kenney toolset.

עבודות שצוטטו

1. Getting started with Asset Forge - Kenney, 2026 ,31 נרשמה גישה בתאריך ינואר, https://kenney.nl/knowledge-base/asset-forge/getting-started-with-asset-forge
2. Importing custom blocks in Asset Forge - Kenney, ,31 נרשמה גישה בתאריך ינואר 2026, https://kenney.nl/knowledge-base/asset-forge/importing-custom-blocks-in-asset-forge
3. Importing Asset Forge Art Assets into Unreal Engine - UE Casts, נרשמה גישה בתאריך ינואר 31, 2026, https://uecasts.com/courses/asset-forge-and-unreal-engine/episodes/importing-asset-forge-art-assets-into-unreal-engine
4. Kenney Shape, 2026 ,31 נרשמה גישה בתאריך ינואר, https://kenney.nl/tools/kenney-shape
5. Kenney Shape documentation, 2026 ,31 נרשמה גישה בתאריך ינואר, https://kenney.nl/knowledge-base/tools/kenney-shape-documentation
6. How do colors work in Kenney &... - Gamedev Mastodon, נרשמה גישה בתאריך ינואר 31, 2026, https://mastodon.gamedev.place/@kenney/112153581016142577
7. Prototype Textures by Kenney (Assets) - Itch.io, ,31 נרשמה גישה בתאריך ינואר 2026, https://kenney-assets.itch.io/prototype-textures
8. Trying to paint parts of an imported .obj - Basics & Interface - Blender Artists Community, 2026 ,31 נרשמה גישה בתאריך ינואר, https://blenderartists.org/t/trying-to-paint-parts-of-an-imported-obj/1579779
9. Blender for Beginners: How to Split an STL or Mesh Object Using the Bisect Tool - YouTube, 2026 ,31 נרשמה גישה בתאריך ינואר, https://www.youtube.com/watch?v=_-U8u5hK_D4
10. 3 Ways To Add Color To Models - FPS Game With Unity & Blender - YouTube, 2026 ,31 נרשמה גישה בתאריך ינואר, https://www.youtube.com/watch?v=7TEcZLIVvHg
11. How do I split a static mesh into multiple parts? - Asset Creation - Unreal Engine Forums, 2026 ,31 נרשמה גישה בתאריך ינואר, https://forums.unrealengine.com/t/how-do-i-split-a-static-mesh-into-multiple-parts/500394
12. Asset Forge (extremely easy-to-use 3D modeling and 2D sprite tool) now available! - Reddit, 2026 ,31 נרשמה גישה בתאריך ינואר, https://www.reddit.com/r/gamedev/comments/6kw6kl/asset_forge_extremely_easytouse_3d_modeling_and/